Artificial Intelligence and Law (2025) 33:519–549
https://doi.org/10.1007/s10506-024-09394-x

ORIGINAL RESEARCH

# Legal sentence boundary detection using hybrid deep learning and statistical models

**Reshma Sheik**[1] · **Sneha Rao Ganta**[1] · **S. Jaya Nirmala**[1]

Accepted: 13 February 2024 / Published online: 14 March 2024
© The Author(s), under exclusive licence to Springer Nature B.V. 2024, corrected publication 2024

## Abstract
Sentence boundary detection (SBD) represents an important first step in natural language processing since accurately identifying sentence boundaries significantly impacts downstream applications. Nevertheless, detecting sentence boundaries within legal texts poses a unique and challenging problem due to their distinct structural and linguistic features. Our approach utilizes deep learning models to leverage delimiter and surrounding context information as input, enabling precise detection of sentence boundaries in English legal texts. We evaluate various deep learning models, including domain-specific transformer models like LegalBERT and CaseLawBERT. To assess the efficacy of our deep learning models, we compare them with a state-of-the-art domain-specific statistical conditional random field (CRF) model. After considering model size, F1-score, and inference time, we identify the Convolutional Neural Network Model (CNN) as the top-performing deep learning model. To further enhance performance, we integrate the features of the CNN model into the subsequent CRF model, creating a hybrid architecture that combines the strengths of both models. Our experiments demonstrate that the hybrid model outperforms the baseline model, achieving a 4% improvement in the F1-score. Additional experiments showcase the superiority of the hybrid model over SBD open-source libraries when confronted with an out-of-domain test set. These findings underscore the importance of efficient SBD in legal texts and emphasize the advantages of employing deep learning models and hybrid architectures to achieve optimal performance.

**Keywords** Natural language processing · Sentence boundary detection · Deep learning · Transformer · LegalBERT · CaseLawBERT · CNN · CRF

✉ Reshma Sheik
rezmasheik@gmail.com

Sneha Rao Ganta
sneharao2303@gmail.com

S. Jaya Nirmala
sjaya@nitt.edu

[1] National Institute of Technology, Trichy, Tiruchirappalli, Tamil Nadu, India

## 1 Introduction

Sentence Boundary Detection (SBD) is an essential component of Natural Language Processing (NLP), which involves identifying the boundaries between sentences in a given text. The proper delineation of sentence boundaries is crucial in any NLP application, as incorrect boundaries can greatly distort the NLP pipeline for many downstream applications (Savelka et al. 2017). While humans can easily identify and separate sentences within a given text, developing automatic sentence boundary detection algorithms is a complex task in NLP (Păiş and Tufiş 2021). The task has been extensively studied in general domains (Read et al. 2012), but detecting sentence boundaries in specialized corpora such as legal text poses significant challenges. The legal text is characterized by lengthy and intricate sentences with lists, citations, abbreviations, and law-specific keywords, often making it difficult to compose actual sentences from legal documents (Savelka et al. 2017). In addition, legal documents of the same kind, like laws or court decisions, can vary significantly in structure and content (Allen and Lysaght 2015), making it difficult to separate one sentence from another. This is where a sentence split system comes in, which must solve these problems by identifying potential boundary spots and accurately pinpointing sentence-ending markers. Therefore, developing accurate SBD algorithms for legal text is a critical research area in NLP and useful in many applications like legal argument mining (Habernal et al. 2023; Chen et al. 2022), legal text summarization (Jain et al. 2023; Deroy et al. 2023), rhetorical role segmentation (Saravanan and Ravindran 2010; Bhattacharya et al. 2021), legal information retrieval (Chalkidis and Androutsopoulos 2017; Mandal et al. 2021) etc.

One of the primary concerns with SBD for legal documents is the ambiguous nature of delimiters involved in it Allen and Lysaght (2015). For example, the period delimiter may indicate the boundary of a line, an abbreviation, a numerical value, or part of a citation in legal text (Sheik et al. 2022). To achieve this goal, a sophisticated algorithm is needed to identify and categorize the diverse range of punctuation marks employed in legal documents while considering their contextual usage. The motivation for this work is to use a deep learning framework that utilizes the contextual information surrounding delimiters for SBD, treating it as a sequence classification problem in English legal text. Furthermore, a performance improvement is demonstrated using a hybrid architecture incorporating the state-of-the-art statistical Conditional Random Field (CRF) model and the Convolutional Neural Network (CNN). The performance of this hybrid model is also tested in the Indian domain and compared with the standard open-source libraries

*Key Contributions* The contributions of this paper can be summed up as follows:

- *Deep Learning Approach* The study uses deep learning architectures, including domain-specific transformer models like LegalBERT and CaseLawBERT, to classify the delimiter as a true sentence end marker. The framework leverages the delimiter and its surrounding context as input to the model for this sequence classification task.

- *Evaluation and Model Selection* The performance of various deep learning models is compared with the state-of-the-art domain-specific CRF model based on model size, F1 score, and inference time. The best-performed CNN model is chosen for the hybrid architecture.
- *Hybrid Architecture* To further enhance performance, the paper proposes a hybrid architecture that combines the strengths of the top-performing CNN model and the subsequent CRF model. By incorporating the features of the CNN model into the CRF model, the hybrid architecture achieves superior results compared to existing baselines.
- *Out-of-Domain Evaluation* Empirical evidence demonstrating that our hybrid model consistently outperforms standard toolkits when applied to an Indian legal dataset. This finding showcases the generalization capabilities of our hybrid architecture in accurately detecting sentence boundaries within the specific context of Indian legal text, even without prior exposure or training on the dataset.

The rest of this paper is arranged as follows. Section 2 presents the literature review of the related works. Section 3 presents the dataset and the legal sentence structure. The deep learning model architecture and hybrid architecture are presented in Sect. 4. Additional experimentation for training and fine-tuning deep learning models is provided in Sect. 5. Section 6 presents the results and discusses the findings. Finally, Section 7 concludes this paper with future works.

## 2 Related work

The SBD has been extensively studied in the NLP literature, and several algorithms and models have been proposed to identify sentence boundaries in a generic corpus (Read et al. 2012). However, these algorithms and models face challenges when dealing with a specialized domain. To address this, our literature review is organized into three sections. Firstly, we examine the literature on SBD in general. Next, we explore works focused on specialized scientific, medical, and finance domains. Lastly, we present existing research pertaining to the legal domain, which possesses distinct language, structure, and content.

### 2.1 Generic sentence boundary detection

In the subsequent sub-sections, we categorize the existing literature into rule-based, supervised, and unsupervised approaches, providing a comprehensive classification of the SBD methods.

#### 2.1.1 Rule-based models

Sentence boundary detection is performed by various systems using hand-crafted rules and regular expressions. These approaches are favored because the errors they produce can be easily understood, and the rules can be adjusted gradually. One of

the earliest rule-based systems, described by Grefenstette and Tapanainen (1994), utilized a set of regular expressions to differentiate periods used in abbreviations, numbers, email addresses, and web addresses from those indicating the end of a sentence. However, this system solely focused on periods as sentence boundaries and achieved an accuracy of 93.78%. Later (Mikheev 2000) treats sentence segmentation with rules to determine whether the words adjacent to a potential sentence boundary are abbreviations or proper names. They adopted a POS tagging approach to sentence splitting, producing highly portable models across different corpora. A more recent rule-based approach—PySBD (Pragmatic Sentence Boundary Disambiguation) (Sadvilkar and Neumann 2020) utilizes a combination of exceptions and regular expression rules. It has proven to be a robust, open-source solution for sentence boundary detection in 22 languages. However, rule-based approaches present challenges when scaling to multiple languages since they require extensive manual effort for each language. Consequently, many studies in sentence boundary detection have moved away from rule-based techniques due to their lack of sustainability and efficiency to the unseen corpus.

#### 2.1.2 Supervised models

In early SBD research, an approach proposed by Riley (1989) utilized decision trees to analyze the contextual lexical features surrounding periods, determining whether they indicate sentence boundaries. This method incorporated statistical features, including word probabilities derived from a 25 million-word corpus, achieving an impressive accuracy of 99.8% on the widely recognized Brown Corpus (Francis and Kucera 1979). Subsequently, the SATZ architecture introduced by Palmer and Hearst (1997) employed either a decision tree or a neural network, leveraging POS features. The SATZ architecture achieved an accuracy of 98.5% when evaluated on the Wall Street Journal (WSJ) dataset. Another statistical system, Reynar and Ratnaparkhi (1997), utilized a maximum entropy classifier that considered simpler lexical features of the words preceding and following potential delimiters and attained accuracy scores of 98.8% and 97.9% on both the WSJ text and the Brown Corpus, respectively. In the study, Wong and Chao (2010) introduced an incremental algorithm that employed various features, including capitalization, word length, potential punctuation, and the presence of specific signs like '$' and numbers. These features were extracted from trigram contexts within a training corpus to detect sentence boundaries. Remarkably, their approach obtained 99.98% accuracy when performed on the Brown corpus.

Following a similar approach, Splitta (Gillick 2009) further improved the method by employing Support Vector Machines (SVM) with lexical features. Splitta concentrated on the specific task of identifying sentence boundaries using periods and demonstrated enhanced performance. However, the aforementioned approaches restricted sentence boundaries to punctuation marks, excluding non-punctuation characters from consideration. The SpaCy library (Honnibal et al. 2020) addressed this limitation by jointly learning dependency parsing and sentence segmentation using a transition-based parser on annotated data. SpaCy did not treat punctuation marks as special entities, allowing for more flexible sentence boundary detection.

### 2.1.3 Unsupervised and self-supervised models

Unsupervised methods aim to acquire segmentation knowledge solely from the raw, unsegmented text. Punkt (Kiss and Strunk 2006) is a completely unsupervised system that recognizes abbreviations, initials, and ordinal numbers. It introduces a language-agnostic, unsupervised technique and emphasizes that abbreviations significantly cause ambiguity in detecting sentence boundaries. To construct a highly accurate abbreviation detector, Punkt utilizes collocation detection (Dunning 1994). In recent advancements, Ersatz (Wicks and Post 2021) introduced a self-supervised model by modernizing the approach proposed by Riley (1989) incorporating a transformer architecture (Vaswani et al. 2017) that considered sub-words as contextual information around punctuation marks. This facilitates training without labeled data, as line breaks inherently divide the text into paragraphs. In contrast to this method, a recently developed WtP model (Minixhofer et al. 2023) empowers self-supervised sentence segmentation without depending on punctuation or making assumptions specific to a particular language. The versatility of this approach was demonstrated in five different sizes, covering a wide range of 85 languages.

However, in specialized domains, the task of SBD is notably influenced by domain-specific data and characteristics. This is primarily because such domains frequently demonstrate deviations from conventional sentence structures, variability in sentence lengths, and the utilization of non-standard punctuation. The authors in Read et al. (2012) note a decline in performance when transitioning from general corpora to domain-specific corpora. Their findings indicate that incorporating a reasonable level of analysis of document structure contributes to increased overall SBD performance.

## 2.2 Domain-specific sentence boundary detection

The initial system developed by Reynar and Ratnaparkhi (1997) focused on achieving superior performance within a specific domain. This model utilized features specifically designed for the English financial language, such as identifying honorifics and corporate designations. Rather than treating sentence segmentation as a standalone task, the approach framed it as a disambiguation task involving targeted delimiters. Previous work limited evaluation on datasets such as the Brown corpus and WSJ and discussed issues with SBD while assuming characteristics of high-quality text, such as appropriate formatting, grammatically correct sentences, and the absence of spelling errors. The work (Griffis et al. 2016) evaluated widely used NLP toolkits to assess their performance on SBD tasks using clinical domain corpora. The study by López and Pardo (2015) primarily focused on SBD in informal content generated by users found across various online platforms. The work by Rudrapal et al. (2015) specifically conducted research on SBD within the context of social media. Moreover, there has been considerable interest in SBD for speech transcriptions, as automated recognition systems often rely on identifying sentence segments within these transcripts. Recent studies (Wang et al. 2019; Rehbein et al.

2020) have investigated the challenges of SBD in speech, which lacks both punctuation and the structured nature of the written text.

### 2.3 Legal sentence boundary detection

The task of sentence boundary detection specific to the legal domain commenced with creating a dataset for US Adjudicatory decisions (Savelka et al. 2017) and explored the use of CRF models (Lafferty et al. 2001). An aggressive tokenization technique was employed, and basic textual features such as token length, digits, spaces, upper/lower case, etc., were considered. These features were utilized to develop multiple CRF models, which resulted in increased inference time. In a related investigation (Sanchez 2019), the identical dataset was investigated using Punkt, CRF, and Bidirectional LSTM neural network architecture. Enhancements were made to the Punkt model by training it with an updated set of abbreviations tailored for the legal text-domain. The neural network architecture employed word-2vec embeddings and a stacked BiLSTM network with a softmax layer for output. However, this classification architecture did not outperform the performance of the CRF model. The following research (Sheik et al. 2022) has explored the application of deep learning models for SBD in the legal domain. Inspired by previous work (Schweter and Ahmed 2019), finding sentence boundaries is treated as a classification problem, where models are trained to identify potential sentence-ending period markers based on the surrounding context. A fixed-size context window consists of concatenating the left and right contexts of the period symbol as input to the model. The CNN model outperformed other deep learning models and the pySBD model. In contrast, the trained CNN model showcased satisfactory performance without necessitating extensive feature engineering while also exhibiting significantly faster inference times compared to state-of-the-art CRF models (Sanchez 2019). However, a primary limitation of the CNN model lies in its sole reliance on the period symbol as a delimiter, necessitating further exploration into other punctuation marks that indicate sentence boundaries.

Our research aims to further enhance the context-specific architecture by incorporating additional delimiters into the model. To achieve this, we introduce the context and the delimiter as input to the models, allowing for a more comprehensive understanding of the sentence segmentation task. In addition to processing English text, Glaser et al. (2021) curated a German dataset, while an even more recent development by Brugger et al. (2023) introduced a diverse legal dataset specifically designed for multilingual models, encompassing six distinct languages. The literature in the legal domain pertaining to models, datasets, and other findings is summarized in Table 1.

Furthermore, it is worth noting that the existing literature in this domain has not yet explored the utilization of hybrid models that combine the deep learning CNN framework with the statistical CRF model. Previous SBD approaches in the legal domain mainly focused on statistical approaches, which rely on word tokenization and other feature engineering tasks. Our proposed neural network models tackle the problem as a sequence classification task treating the character context surrounding

**Table 1** Literature on sentence boundary detection task in the legal domain

| Sl. No. | References | Models used | Findings | Dataset |
|---|---|---|---|---|
| 1 | Savelka et al. (2017) | CRF model | –Introduced the problem of SBD in English Domain.<br>–CRF model performed better than the traditional SBD systems. | US adjudicatory decisions |
| 2 | Sanchez (2019) | Punkt, CRF and BiLSTM | –Used unsupervised model with custom abbreviations and neural network model.<br>–CRF surpasses all the models. | US adjudicatory decisions |
| 3 | Sheik et al. (2022) | CNN, LSTM, GRU, Transformer | –SBD task as a sequence classification task for identifying the period as a true boundary marker.<br>–CNN showed better performance based on model size and inference time. | US adjudicatory decisions |
| 4 | Glaser et al. (2021) | CRF, BiGRU | –Addressed the issue of SBD within the German domain.<br>–Neural Network and CRF models performed better when compared with standard libraries. | German legal documents |
| 5 | Brugger et al. (2023) | CRF, BiLSTM-CRF, Transformer | –Curated a multilingual dataset and models for the SBD task.<br>–Extracted word embeddings for training the models. | Italian, French, Spanish, English, German, Portuguese |

**Table 2** Statistics of the English dataset on two domains

| Domain | Decision | #Chars | #Tokens | #Sentences | #Average Tokens/Sentence |
|---|---|---|---|---|---|
| US | IP | 932,133 | 343,831 | 7262 | 21.42 |
| | BVA | 474,478 | 170,166 | 3727 | 20.73 |
| | SC | 960,890 | 31,872 | 602 | 24.2 |
| | CC | 984,756 | 367,740 | 8295 | 19.82 |
| Indian | SC | 971,959 | 170,086 | 5640 | 30.15 |

the delimiters as input. As much of the legal text is digitized via Optical Character Recognition (OCR) systems, introduces inaccuracies in word boundaries, making the character-based approach more suitable compared to word-based approaches. Moreover, our innovation extends to the introduction of a hybrid model, integrating CNN output features into the CRF model. This hybridization enhances the predictive capabilities of our SBD system, addressing limitations observed in prior models and offering improved accuracy in identifying sentence boundaries. This approach holds great potential for improving the performance and accuracy of sentence segmentation within the legal domain, addressing the unique challenges and complexities it presents.

## 3 Dataset

The work employed a dataset derived from Savelka et al. (2017), which consists of 80 court decisions from four distinct domains: Cyber Crime (CC), Intellectual Properties (IP), Board of Veterans (BVA), and the United States Supreme Court (SC). These decisions were stored in four JSON (JavaScript Object Notation) files, with each file containing 20 decisions accompanied by a list of offsets marking the sentence boundaries. This data resource[1] comprising 26,052 annotated sentences is publicly available for research purposes.

We curated another English legal dataset for evaluating our model from the Indian domain. Legal terminology used in Indian legal documents often includes phrases and terms specific to the Indian legal system. These terms may possess different meanings or interpretations compared to legal terms used in other jurisdictions. This distinction requires careful consideration and examination when analyzing Indian legal datasets. The dataset (Malik et al. 2021) for the explanation in model prediction for 56 Indian Supreme Court (SC) judgements was used for this purpose. Table 2 presents the detailed statistics of the two datasets. The statistics clearly indicate that the average number of tokens per sentence in the Indian context is higher compared to the US domain.

[1] https://github.com/jsavelka/sbd_adjudicatory_dec.

### 3.1 Legal sentence structure

In legal documents, sentences generally adhere to a standard structure, encompassing a subject, object, and verb in the correct grammatical order. Punctuation marks like periods, semicolons, question marks, or colons typically terminate these sentences (Păiş and Tufiş 2021). Legal language is characterized by many long compounds and *complex sentences*, often comprising multiple clauses contributing to their *extensive length*. There have been documented instances of sentences in a legal language containing as many as 494 words, underscoring the long and complex nature of this linguistic style (Lavery 1921). Complexity in legal sentences arises from several factors, including excessive modifications, over-caution, the use of embedded clauses, atypical word order, and the inclusion of words or phrases in unconventional contexts that deviate from typical discourse patterns.

*Headlines* play a prominent role in legal documents, establishing the text's structure and highlighting relationships between different parts. It's important to note that certain headlines may incorporate case names, adding complexity to their interpretation. *Citations*, commonly found in judgments, serve to connect various cases, statutes, or laws. *Lists* are a noticeable structural component in legal documents that often pose challenges in determining their sentence boundaries. For instance, in enumerated lists, each item of the enumeration is treated as an individual sentence to ensure consistency within the list (Savelka et al. 2017). These lists can be embedded within sentences featuring a variable middle part, where different subjects or objects are possible for a clause. *Data fields* containing information such as dates, people, locations, and case names require special attention during analysis. *Endnotes/footnotes* should also be addressed effectively. Legal documents can exhibit mixed cases involving the aforementioned structures. Moreover, even within the same legal document type, there exists significant variation (Allen and Lysaght 2015), including instances of parentheses within sentences. Figure 1 illustrates a few examples of intricate and lengthy sentence structure regarding the use of the English language in the Indian legal domain.

### 3.2 Data processing

All deep learning models were trained using BVA, IP, and SC decisions and evaluated using CC decisions. Once the end-marker symbol is identified within the input text, the character-level context surrounding it is extracted and used as input for the deep-learning model. In the case of the transformer architecture, we acquire contextual information at the sub-word level. Thus, to identify potential end-of-sentence markers, the models effectively capture character-level information. Table 3 illustrates sample inputs and their corresponding left and right contexts.

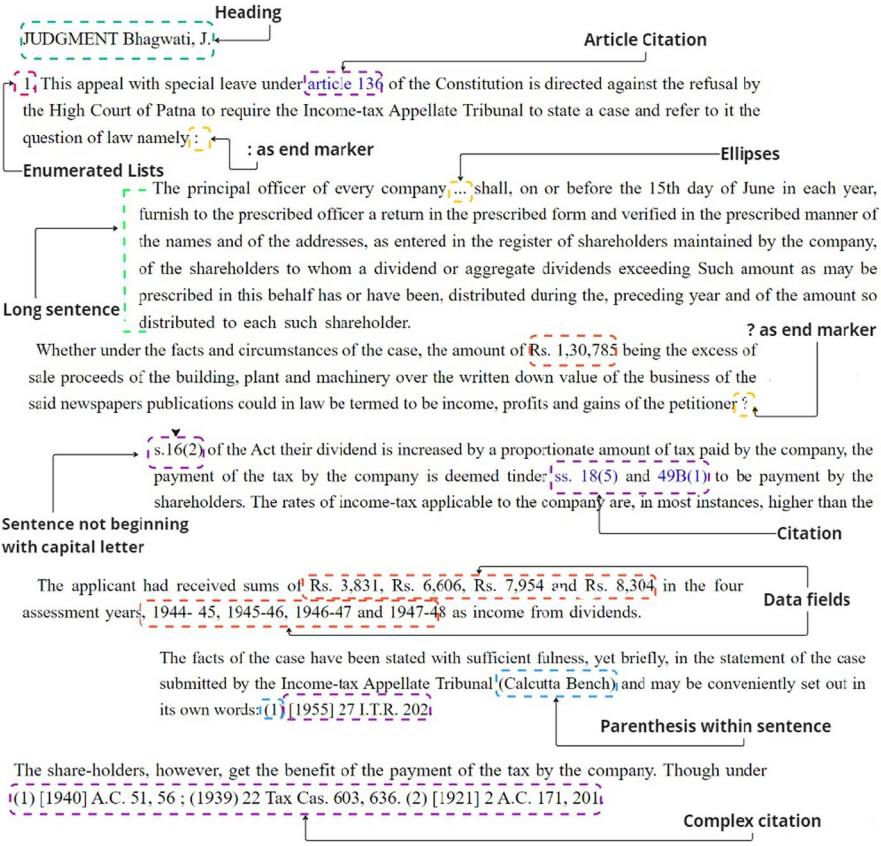


**Fig. 1** Legal sentence structure: sample sentences extracted from Indian judicial decisions

**Table 3** Sample inputs fed to the model for training

| Left context | Delimiter | Right context | Sample input |
|---|---|---|---|
| ‘aboard’ | . | ‘[The’ | ‘aboard. [The’ |
| ‘at 408’ | ; | ‘See a’ | ‘at 408; See a’ |
| ‘il 197’ | , | ‘and 3’ | ‘il 197, and 3’ |
| ‘bility’ | ( | ‘TDIU).’ | ‘bility(TDIU).’ |
| ‘2008.’ | * | ‘1155 D’ | ‘2008.*1155 D’ |
| ‘fendan’ | - | ‘appell’ | ‘fendan-appell’ |
| ‘U.S.C.’ | § | ‘1029(’ | ‘U.S.C.§1029(’ |
| ‘manner’ | " | ‘and a’ | ‘manner" and a’ |
| ‘ate: 0’ | / | ‘13/15’ | ‘ate: 0/13/15’ |
| ‘OLLAND’ | [ | ‘*] Dis’ | ‘OLLAND[*] Dis’ |
| ‘hat "[’ | ] | ‘f the’ | ‘hat "[]f the’ |
| ‘2B1.1(’ | ) | ‘(10),’ | ‘2B1.1()(10),’ |
| ‘s? ATM’ | ? | ‘Drivi’ | ‘s? ATM? Drivi’ |

## 4 Model architecture

### 4.1 Deep learning models

The employed deep learning models adopt a context window-based approach to process legal documents. Algorithm 1 shows the entire steps for the context window-based approach for the SBD task. For each delimiter in the input document, extract the left and right contexts, denoted as $L$ and $R$, respectively. Then, obtain the optimized window for the left and right contexts, represented as $L_{\text{opt}}$ and $R_{\text{opt}}$. These contexts along with the delimiter serve as input to the embedding layer, resulting in embeddings $E$. These embeddings are fed into the deep learning model framework, denoted as $DL$, which produces the output vector $O$. The output vector $O$ is passed through a dense layer with sigmoid activation, represented by the function $\sigma$, to obtain the classification output $Y$. If $Y \geq 0.5$, the delimiter in the legal text signifies the end of a sentence. Otherwise, it does not signify the end of a sentence.

**Algorithm 1** Context Window-Based Deep Learning Approach for SBD task

**Input:** Legal document
**Output:** Binary classification output
**for** each delimiter in the legal document **do**
    $L, R \leftarrow$ extract_contexts(Delimiter)
    $L_{\text{opt}} \leftarrow L[-\text{Window size}:]$
    $R_{\text{opt}} \leftarrow R[:\text{Window size}]$
    $E \leftarrow$ embedding_layer($L_{\text{opt}}$,Delimiter, $R_{\text{opt}}$)
    $O \leftarrow \text{DL}(E)$
    $Y \leftarrow \sigma(\text{dense_layer}(O))$
    **if** $Y \geq 0.5$ **then**
        **Output:** True // Delimiter signifies end of sentence
    **else**
        **Output:** False // Delimiter does not signify the end of sentence
    **end if**
**end for**

We constructed five distinct neural network architectures: Long Short Term Memory (LSTM) (Gers et al. 2000), Gated Recurrent Unit (GRU) (Chung et al. 2014), Bidirectional LSTM (BiLSTM), Bidirectional GRU (BiGRU), and Convolutional Neural Network (CNN). Additionally, the BiLSTM and BiGRU models were trained with attention mechanisms (Bahdanau et al. 2015), enabling them to focus on key segments within the input. The following subsection briefly describes the deep learning models used in the implementation.

#### 4.1.1 LSTM

For our experiments, we utilize a standard LSTM network with an embedding dimension of 128 and a hidden dimension of 256. To introduce some regularization,

we apply a dropout probability of 0.2 to the hidden layer. The resulting output is then fed into a dense layer with a sigmoid activation function for classification.

#### 4.1.2 BiLSTM

To enhance the contextual understanding, we employ a Bidirectional LSTM structure that handles input in both the forward and backward directions. This results in an input size of 512 when passed to the following dense layer. The remaining parameters and configurations remain comparable to the LSTM design. Later, we introduce attention weights to integrate an attention technique to the BiLSTM architecture (Lin et al. 2017). Including attention determines the soft alignment scores between each hidden state and the final hidden state of the LSTM. This enables the model to capture the comprehensive interconnections between the input elements and the output.

#### 4.1.3 GRU

The GRU architecture employs 256 hidden states and an embedding size of 128. A dropout probability of 0.2 is applied after the hidden layer during training. GRU is a more compact variant of LSTM, as it does not feature an explicit memory unit.

#### 4.1.4 BiGRU

Similar to the BiLSTM, the BiGRU architecture shares the same design but utilizes GRU instead of LSTM. By incorporating attention into the BiGRU architecture, the model allows for interactions between the inputs to determine which parts deserve heightened attention. These dynamic interactions, in conjunction with attention scores, are subsequently synthesized to generate the final output representations.

#### 4.1.5 CNN

Figure 2 shows our CNN architecture, which employs a 1D convolutional layer encompassing six filters with a kernel size of five. The resulting output from the convolutional filter is concatenated to construct a comprehensive feature vector, subsequently propagated through a global max pooling layer. Prior to the prediction layer, a 250-dimensional hidden layer employing ReLU activation (Agarap 2018) is employed. During training, the model employs a learning rate of 0.001 and incorporates dropout with a probability of 0.2.

### 4.2 Transformer models

We fine-tuned two domain-specific pre-trained transformer models for the task of SBD. The details of their architecture are outlined in the sections below.

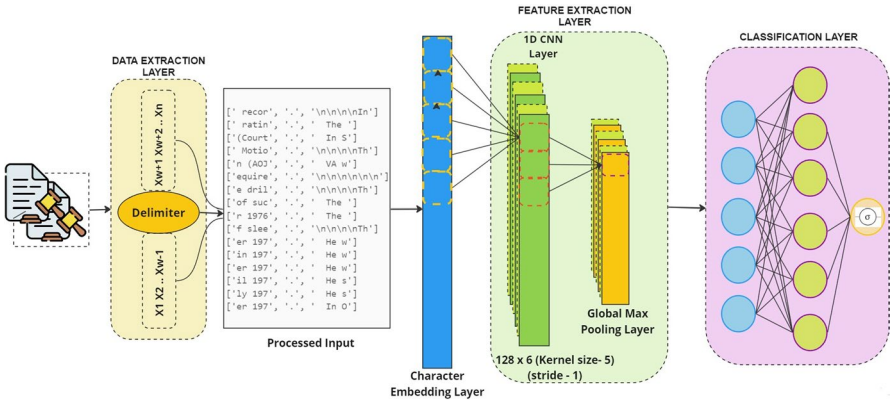


**Fig. 2** Context-based sentence boundary detection using convolutional neural network

### 4.2.1 LegalBERT

LEGAL-BERT is a suite of BERT models specifically developed to facilitate legal NLP research. The work (Chalkidis et al. 2020) explores three distinct options of LegalBERT for domain adaptation: (i) utilizing the original BERT model without any modifications, (ii) augmenting pre-training with additional domain-specific corpus, and (iii) training BERT from scratch using domain-specific data. These models were trained on datasets from EU laws, ECHR cases, and US contracts. Our experiments focus on the “legal-bert-base-uncased" variant, a model trained from scratch on legal texts.

### 4.2.2 CaseLawBERT

CaseLawBERT (Zheng et al. 2021) is a model trained on Harvard Law case corpus[2] encompassing 3,446,187 legal decisions across various federal and state courts. For the training objective, the model begins with the base BERT model, bert-base-uncased (uncased, 110 M parameters), and undergoes further training for an additional 1 M steps. This training is performed on the Masked Language Model (MLM) and Next Sentence Prediction (NSP) objectives.

## 4.3 Statistical CRF model

A CRF model is a probabilistic graphical model used for sequence labeling, capturing dependencies between inputs and outputs in sequential data (Lafferty et al. 2001). The CRF model is trained to learn the probabilities of label sequences given the input sequence using training data. In our experiments, we employed the first-order CRF, where each observation $X_i$ is associated with a label $Y_i$. We represented

[2] https://case.law/.

**Table 4** The feature functions used in the CRF model

| Feature | Description |
|---|---|
| lower | Represents a token that is in lowercase |
| sig | Represents a feature that captures the signature of a token. The signature is generated by applying the following transformations to the token<br>Each lowercase letter is replaced with 'c'<br>Each uppercase letter is replaced with 'C'<br>Each digit is replaced with 'D' |
| length | Represents a feature indicating the token length in characters. If the length is less than four, the feature value is set to the actual length. If the length is between four and six, the value is marked as "normal." For lengths more than six, the feature is marked as "long." |
| islower | Represents a true binary value if all of the token's characters are in lowercase |
| isupper | Represents a true binary value if all the token's characters are in upper case |
| istitle | Represents a true binary value if the first character of the token is in upper case and the rest of the characters are in lower case |
| isdigit | Represents a true binary value if all the characters in the token are digits |
| isspace | Represents a true binary value if the characters in the token are whitespace |

the sequence using tokenized representation along with a list of simple features obtained from Savelka et al. (2017). Table 4 highlights the feature functions employed in the model. It considers the current element's features and the features of its neighboring elements to determine the appropriate label for each element.

Mathematically, we denote the feature vectors as $f_k(Y_i, Y_{i-1}, X_i)$, with $Y_i$, $Y_{i-1}$, and $X_i$ representing the current label, previous label, and current token, respectively. Each feature can take a value of 0 or 1, depending on the presence or absence of the corresponding feature for each token. The CRF assigns weights $w_k$ to these feature functions based on the training data, resulting in a score computed as the sum of the weighted feature values for each token. To convert this score into a probability distribution, we exponentiate and normalize it over all possible feature functions. By multiplying the probabilities of each word, we obtain the final equation 1.

$$P(\mathbf{Y} \mid \mathbf{X}) = \frac{1}{Z(\mathbf{x})} \prod_{i=1}^{n} \exp\left\{ \sum_{k=1}^{K} w_k f_k(Y_i, Y_{i-1}, \mathbf{X}_i) \right\} \tag{1}$$

Here, $P(\mathbf{Y} \mid \mathbf{X})$ represents the probability of the label sequence $Y$ given the input sequence $X$, and $Z(x)$ is the normalization factor that accounts for all possible feature functions. Computing $Z(x)$ exactly can be challenging due to the potentially large number of feature functions. Therefore, we resort to approximate estimation techniques, as mentioned in Sutton et al. (2012). For training our CRF models, we used the CRFsuite toolkit[3] Okazaki (2007), which estimates label probabilities based on observed training examples. The CRF model's BIL (Beginning, Inside, Last) scheme

[3] https://www.chokkan.org/software/crfsuite/.

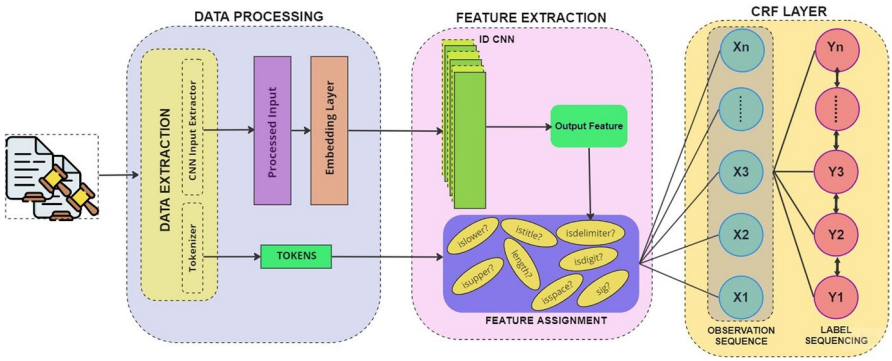


**Fig. 3** Hybrid model: CNN-CRF architecture

is used to label tokens within the sequence. During inference, the model identifies the label sequence with the highest probability for the input sequence.

### 4.4 Hybrid model

In our proposed hybrid architecture, we treated SBD as a sequence labeling problem and used features from the deep learning-based CNN to train the statistical CRF model. CRF model is a custom SBD system, wherein we trained several conditional random field models based on simple low-level features. More complex features can help to improve the performance even further, as is evident from the literature that the inclusion of special tokens is important for good performance in Law documents (Glaser et al. 2021). The CNN model is adept at capturing intricate patterns and features from the input data, making it particularly effective for tasks such as feature extraction in NLP (Chollampatt and Ng 2018). In our hybrid setup, the CNN generates valuable insights from the character-level tokenization of legal texts. These extracted features, especially the indication of delimiters, were then integrated into the CRF model. The CRF model, on the other hand, excels in utilizing sequential dependencies and contextual information. By combining the strengths of both models, we can achieve a synergistic effect and increase the performance of our SBD system. Figure 3 presents the overall architecture of the hybrid model. The data processing layer was used to extract essential input for the CNN model and to tokenize the text for the subsequent feature assignment process. The feature extraction layer specifically focused on extracting and incorporating the output feature corresponding to the *isdelimiter* condition from the CNN model. This feature is combined with other relevant features within the CRF model to optimize the effectiveness of feature assignment. Finally, the CRF layer used the assigned features associated with each token to predict the appropriate labels for the observation sequence.

Algorithm 2 illustrates the steps involved in the hybrid model. The CNN-CRF hybrid model predicts label sequences for input text sequences. It tokenizes the input text and extracts the delimiter $D$ and its contextual features using the CNN Input Extractor. An embedding layer obtains the embedded representation $E_D$ of $D$. The

algorithm then employs a 1D CNN layer to capture the output feature of the delimiter token. This feature is fed into a CRF layer, along with the token features, to learn label sequence probabilities using a first-order CRF. The CRF layer predicts the labels for the observation sequence, resulting in the predicted label sequence *Y*.

**Algorithm 2** Hybrid model: CNN-CRF

**Input:** Delimiter $D$, Input text sequence $X$, Training data
**Output:** Predicted label sequence $Y$
Get the Tokenized text $T$ for the input text sequence $X$:
$T = \text{Tokenizer}(X)$.
Use CNN Input Extractor to obtain $D$ and its contextual features $D_context$.
Obtain the embedded representation of $D$:
$E_D = \text{Embedding_Layer}(D_context)$.
Feed $E_D$ into a 1D CNN layer.
Capture the output feature of the delimiter token using the CNN layer:
$O_D = \text{CNN_Layer}(E_D)$.
Feed $O_D$ into a CRF layer.
Assign the final features for the tokens $T$ to feed it to the CRF layer.
Use first-order $CRF$ in experiments to learn probabilities of label sequences using training data.
Predict the labels in the CRF layer for the observation sequence $X_i$:
$Y = \text{CRF_Layer}(X_i)$.
**Output:** Predicted label sequence $Y$

## 5 Implementation details

The implementation was carried out using the Python Language version 3.10.4 and used torch version '1.12.1+cu113' and cuda version '11.5'. The computational power was provided by an NVIDIA GeForce RTX 3060 graphics card on a Windows 10 machine with an Intel Core i7 processor. To facilitate the development process, we leveraged several essential libraries. The Hugging Face library[4] was employed for fine-tuning the pre-trained transformer-based models, while pandas[5] and numpy[6] libraries were utilized for data pre-processing and post-processing tasks. Furthermore, the statsmodel[7] library was employed for conducting statistical tests, and the

[4] https://huggingface.co/.
[5] https://pandas.pydata.org/.
[6] https://numpy.org/.
[7] https://www.statsmodels.org/stable/index.html.

**Table 5** Models and its attributes for training

| Model | #Parameters | #Epochs | Training time/ epoch (in seconds) |
|---|---|---|---|
| LSTM | 4,08,449 | 10 | 1.88 |
| BiLSTM | 8,03,969 | 10 | 2.87 |
| BiLSTM + attn | 8,03,969 | 10 | 2.98 |
| GRU | 3,09,633 | 15 | 1.86 |
| BiGRU | 6,06,337 | 10 | 2.86 |
| BiGRU + attn | 6,06,337 | 10 | 3.06 |
| CNN | **29,275** | 15 | **1.78** |
| LEGALBERT | 110 M | 4 | 128.98 |
| CaseLawBERT | 110 M | 4 | 2010.67 |

matplotlib[8] library enabled visualization and analysis of the obtained results. To enhance the development experience and facilitate debugging, we opted for the Visual Studio Code[9] integrated development environment (IDE). In the sections below, we provide the specific details of the experiments conducted for training the models. Lastly, we include a subsection that outlines the details of the standard open-source toolkits used for evaluation purposes.

### 5.1 Model implementation

The deep learning models underwent training for a maximum of 25 epochs and used early stopping as the convergence criteria. Table 5 displays the optimal number of epochs and the corresponding training time per epoch. Through our experimentation, we found that employing a context size of six surrounding characters yielded improved results. The training process involved using a learning rate of 1e−3, a mini-batch size of 32, and the adam optimizer (Kingma and Ba 2015) with binary-cross entropy as the loss function. For transformer models, the delimiter along with its contextual information at the sub-word level, is fed as input to the encoder block of a transformer. The transformer models were trained for four epochs at a learning rate of 5e−5. As illustrated in Table 5, highlighted in bold, the CNN model exhibit optimal training efficiency when compared to other deep learning models. In the hybrid configuration, the CNN model's "isdelimiter: value of sigmoid" output feature is fused as an additional feature, into the CRF model. Training is conducted utilizing the python-crfsuite[10] framework, leveraging the Limited-memory Broyden–Fletcher–Goldfarb–Shanno (L-BFGS) algorithm (Liu and Nocedal 1989)

[8] https://matplotlib.org/.
[9] https://code.visualstudio.com/.
[10] https://pypi.org/project/sklearn-crfsuite/.

with a maximum of 100 iterations and Elastic Net (L1 + L2) regularization. The details of implementation with the associated datasets are available.[11]

## 5.2 Standard toolkits for evaluation

### 5.2.1 NLTK

The Natural Language Toolkit (NLTK) (Bird 2006) is a comprehensive collection of tools, datasets, and other frameworks that support NLP applications. NLTK is built in Python and made available under the GPL open-source license. It contains simple interfaces for over 50 corpora and lexical resources, such as WordNet, and also includes a set of libraries for processing text.

### 5.2.2 CoreNLP

Stanford CoreNLP (Manning et al. 2014) aims to provide users with a streamlined and efficient method to obtain linguistic annotations for text, minimizing the effort required. By presenting a unified API, CoreNLP uses a rule-based system that leverages punctuations to accurately identify sentence boundaries. Moreover, CoreNLP offers a lightweight framework based on plain Java objects, which allows seamless integration of CoreNLP as a component within larger software systems, empowering users to build sophisticated and comprehensive language processing applications.

### 5.2.3 SpaCy

SpaCy (Honnibal et al. 2020) is an open-source library that is widely used for building information extraction and natural language understanding systems. It also helps in preprocessing text for deep learning tasks. One of its key features is the ability to tokenize text, even in complex cases involving abbreviations and multiple punctuation marks. In the latest version, spaCy v3.0, transformer-based pipelines have been introduced, which greatly enhance the library's accuracy, bringing it in line with the most advanced techniques available.

### 5.2.4 Syntok

The core purpose of the syntok segmenter[12] is to divide token streams generated by the tokenizer into separate sentences specifically for Indo-European languages. Notably, when confronted with sentence terminal markers that lack subsequent spacing characters, syntok skillfully handles this scenario by employing a two-step approach: first tokenization and then segmentation. This method ensures

[11] https://github.com/researchnlp/SBDLegal.
[12] https://pypi.org/project/syntok/.

**Table 6** Summary of standard toolkits for SBD task and their characteristics

| Toolkit | Version | Language | First Release | License |
|---|---|---|---|---|
| NLTK[13] | nltk-3.8.1 | Python | 2001 | Apache 2.0 |
| CoreNLP[14] | corenlp-0.0.14 | Java | 2010 | GNU GPLv3 |
| SpaCy[15] | spacy-3.5.3 | Python | 2015 | MIT license |
| syntok[16] | syntok-1.4.4 | Python | 2018 | MIT license |
| stanza[17] | stanza-1.5.0 | Python | 2019 | Apache 2.0 |
| Blink Fire[18] | blingfire-0.1.8 | Python | 2020 | MIT license |
| pySBD[19] | pysbd-0.3.4 | Python | 2020 | MIT license |

[13]-https://www.nltk.org/

[14]-https://stanfordnlp.github.io/CoreNLP/

[15]-https://spacy.io/api/doc

[16]-https://pypi.org/project/syntok/

[17]-https://stanfordnlp.github.io/stanza/

[18]-https://github.com/microsoft/BlingFire

[19]-https://github.com/nipunsadvilkar/pySBD

efficient and accurate segmentation, effectively addressing potential challenges that might arise in such cases.

### 5.2.5 Stanza

Stanza (Qi et al. 2020) is a Python package for natural language analysis that offers support for 66 human languages. It provides a comprehensive set of tools that can be seamlessly integrated into a pipeline. These tools enable tasks such as converting human language text into sentence and word lists and the design is language-agnostic built on a fully neural architecture. Additionally, Stanza's functionality can be extended using its CoreNLP client, which incorporates supplementary NLP tools. While Stanza's extensive use of accurate neural models may result in slightly longer annotation times compared to spaCy, it remains competitive in terms of accuracy compared to similar toolkits.

### 5.2.6 Blink fire

Bling Fire presents a diverse array of tokenization algorithms, encompassing pattern-based, WordPiece, and SentencePiece (Kudo and Richardson 2018). Its high-level API is designed to minimize configuration, initialization, or additional file requirements, ensuring seamless integration with popular programming languages. Remarkably, Bling Fire's default models demonstrate a substantial speed advantage, delivering equivalent functionalities as SpaCy but with a 10x faster processing rate.

#### 5.2.7 PySBD

PySBD (Sadvilkar and Neumann 2020), a rule-based SBD, excels in handling 22 different languages. It offers a practical segmenter capable of generating coherent sentences even when dealing with texts of unknown format and domain. PySBD demonstrates its proficiency in out-of-domain corpora, showcasing a successful disambiguation of 97% of sentence boundaries and works out-of-the-box for 22 languages.

Table 6 presents the summary of open-source toolkits used for the evaluation of SBD task.

## 6 Results and discussion

In the subsequent subsection, our focus shifts to evaluating the performance of the trained models. We also compare the performance of deep learning models based on model size training and testing time. We used the traditional classification metrics- Precision, Recall, and F1 score for evaluation. The precision is crucial for assessing the accuracy of positive predictions, recall focuses on the model's ability to capture all relevant instances, and the F1 score strikes a balance between the two, making them particularly suitable for scenarios where class imbalances exist. These metrics collectively offer a comprehensive evaluation of the model's performance, addressing both false positives and false negatives in the context of our study. Moreover, to ensure a fair comparison with the previous approaches (Sanchez 2019), we have taken these metrics conducting the evaluation at the micro level computing statistics across all sentences within all decisions present in a given domain. Later, in Sect. 6.2, we compare the performance of our hybrid CNN-CRF model against standard open-source libraries. Finally, we conclude with a discussion section where we analyze and interpret the findings.

### 6.1 Performance of the trained models

The outcomes derived from the neural network frameworks undergo a post-processing phase to assign labels to the tokens. Our focus lies on the *Begin* and *Last* tokens and uses the demarcation mentioned in the base paper (Sanchez 2019). This is substantial as the baseline models are assessed using offset boundaries. In this process, the model predicts labels for the test set, and the document is sentence-tokenized. The sentences in the test files are extracted using offset boundaries, and they are assigned the labels *Begin* and *Last* based on their respective positions in the result.

Table 7 summarizes our deep learning model results with the baseline CRF model. For effective comparison, each model's precision, recall, and F1 score are

**Table 7** Performance of the trained models

| Model | | Precision | | | Recall | | | F1-score | | |
|---|---|---|---|---|---|---|---|---|---|---|
| | | Begin | Last | Avg. | Begin | Last | Avg. | Begin | Last | Avg. |
| Deep Learning | LSTM | 0.809 | 0.865 | 0.837 | 0.809 | 0.861 | 0.837 | 0.809 | 0.861 | 0.835 |
| | BiLSTM | 0.804 | 0.857 | 0.830 | 0.804 | 0.853 | 0.828 | 0.804 | 0.853 | 0.829 |
| | BiLSTM+attn | 0.810 | 0.863 | 0.836 | 0.810 | 0.858 | 0.834 | 0.810 | 0.858 | 0.834 |
| | GRU | 0.809 | 0.863 | 0.836 | 0.809 | 0.858 | 0.834 | 0.809 | 0.859 | 0.834 |
| | BiGRU | 0.807 | 0.859 | 0.833 | 0.807 | 0.855 | 0.831 | 0.807 | 0.855 | 0.831 |
| | BiGRU+attn | 0.802 | 0.854 | 0.828 | 0.802 | 0.850 | 0.826 | 0.802 | 0.850 | 0.826 |
| | CNN | 0.876 | 0.947 | 0.911 | 0.876 | 0.942 | 0.909 | 0.876 | 0.942 | 0.909 |
| | LEGALBERT | 0.790 | 0.813 | 0.801 | 0.790 | 0.804 | 0.797 | 0.79 | 0.804 | 0.797 |
| | CaseLawBERT | 0.793 | 0.830 | 0.811 | 0.793 | 0.821 | 0.807 | 0.793 | 0.821 | 0.807 |
| Statistical | CRF Sanchez (2019) | 0.923 | 0.928 | 0.925 | 0.867 | 0.859 | 0.863 | 0.894 | 0.892 | 0.893 |
| Hybrid | CNN_CRF | 0.940 | 0.933 | 0.936 | 0.940 | 0.933 | 0.936 | 0.940 | 0.933 | **0.936** |

**Table 8** Model size, training, and inference time of deep learning models

| Model | Model size | Total training time (in seconds) | Total inference time (in seconds) |
|---|---|---|---|
| LSTM | 1.55 MB | 18.8 | 0.85 |
| BiLSTM | 3.21 MB | 28.7 | 1.75 |
| BiLSTM + attn | 3.21 MB | 29.8 | 1.71 |
| GRU | 1.18 MB | 27.9 | 0.62 |
| BiGRU | 2.31 MB | 28.6 | 1.19 |
| BiGRU + attn | 2.31 MB | 76.5 | 1.31 |
| CNN | **116 kb** | 26.7 | **0.16** |
| LEGAL-BERT | 418 MB | 515.93 | 116 |
| CaseLAWBERT | 420 MB | 8042.7 | 113 |

**Table 9** Hybrid model performance against standard libraries

| Model | | Precision | | | Recall | | | F1-score | | |
|---|---|---|---|---|---|---|---|---|---|---|
| | | Begin | Last | Avg. | Begin | Last | Avg. | Begin | Last | Avg. |
| Standard Toolkits | NLTK | 0.462 | 0.476 | 0.467 | 0.462 | 0.471 | 0.466 | 0.462 | 0.471 | 0.4665 |
| | spaCy | 0.500 | 0.522 | 0.511 | 0.500 | 0.515 | 0.507 | 0.500 | 0.516 | 0.508 |
| | stanza | 0.538 | 0.540 | 0.539 | 0.538 | 0.532 | 0.535 | 0.538 | 0.533 | 0.5355 |
| | corenlp | 0.416 | 0.555 | 0.485 | 0.443 | 0.588 | 0.515 | 0.502 | 0.494 | 0.498 |
| | syntok | 0.506 | 0.490 | 0.498 | 0.506 | 0.230 | 0.368 | 0.506 | 0.242 | 0.374 |
| | blingfire | 0.602 | 0.582 | 0.592 | 0.602 | 0.580 | 0.591 | 0.602 | 0.58 | 0.591 |
| | pySBD | 0.797 | 0.817 | 0.807 | 0.797 | 0.807 | 0.802 | 0.797 | 0.807 | 0.802 |
| Hybrid | CNN_CRF | 0.876 | 0.898 | 0.887 | 0.872 | 0.898 | 0.885 | 0.872 | 0.898 | **0.885** |

calculated at the *Begin* and *Last* token levels. We present the outcomes of our hybrid model, which exhibits the best average F1 score highlighted in bold, in comparison to other trained models. Table 8 displays the model size, training, and inference time of the models for the same hardware specifications, wherein the best model size and inference time have been highlighted in bold.

### 6.2 Out-of-domain evaluation

Out-of-domain evaluation refers to evaluating a trained model's performance on a dataset that was not encountered during training. Table 9 displays a comparative analysis of the hybrid model's performance against standard open-source libraries on the Indian dataset. The value highlighted in bold shows its superior performance against other standard toolkits.

### 6.3 Discussion

Deep learning models have transformed various NLP applications, showing significant differences in performance. The study by Collobert et al. (2011) demonstrated that a simple deep learning framework outperforms most state-of-the-art approaches in several NLP tasks. For our task, which involves sequence classification, we sought to find the best deep learning model, starting from Recurrent Neural Networks (RNNs) and progressing to more powerful transformer models. RNNs were the initial choice due to their sequential processing capabilities, but studies (Chung et al. 2014) demonstrated the superiority of gated units like LSTM and GRU over traditional RNNs. We modified the basic framework of these gated RNNs by adding bidirectional and attention techniques. Additionally, we explored the use of CNN for sentence modeling. As RNNs aim to build a composition of arbitrarily long sentences, the CNNs focus on extracting essential n-gram features from input sentences, generating valuable latent semantic representations that prove advantageous for downstream tasks (Young et al. 2018).

The results presented in Table 7 highlight the superior performance of the CNN model over other neural network architectures, as well as the CRF model. Remarkably, when compared to other deep learning models, Table 5 demonstrates that CNN models exhibit shorter training times, while transformer models require nearly 40 times longer training. Also, domain-specific transformer models require more training data in order to achieve substantial results. LEGAL-BERT and CaselawBERT models are challenging to implement due to their complex attention mechanism and large model size. Moreover, due to the lack of substantial training data and taking into account the use of input embeddings at the sub-word level, we could see a loss in performance. Furthermore, it is important to highlight that the CNN model exhibits the fastest inference time (see Table 8), surpassing even the CRF models, mainly due to the extensive feature engineering inherent in the CRF model architecture. This efficiency in training and inference times positions the CNN model as the best model with less computational cost for the proposed hybrid architecture. These findings provide strong support for the rationale behind incorporating the CNN model into our hybrid framework.

We employed the McNemer Statistical test (McNemar 1947) to compare the performance of the top two classifiers, the statistical CRF and deep learning CNN. The test is based on a $2 \times 2$ contingency table that shows the frequency distribution of the outcomes of the two models' predictions on the same dataset. The null hypothesis for McNemar's test assumes that the two models have the same performance and contribute to the same type of errors. The alternative hypothesis is that there is a difference in their performance. McNemar's test calculates a chi-square statistic that follows a chi-square distribution with one degree of freedom (Dietterich 1998). The $p$-value associated with the chi-square statistic is then compared to a chosen significance level (e.g., 0.05) to determine the statistical significance of the observed differences. The obtained $p$-value is way below this significance level, which suggests that there is a significant difference in performance between the two models, rejecting the null hypothesis. Table 10 shows examples of the errors generated by the models. The SBD analysis revealed several common errors shared by the CNN and CRF models. Notably, both

**Table 10** Sample errors generated by the trained CNN, CRF and the Hybrid models in the context of US legal domain

| Sl. No.: | Model | Sample errors |
|---|---|---|
| 1 | CRF | *Ground Truth:* See, e.g., Cal. Family Code Ann. §760 (West 2004). |
| | CNN | *Predicted:* 4 See, e.g., Cal. |
| 2 | CRF | *Ground Truth:* Davina T. Chen, Deputy Federal Public Defender, Los Angeles, CA, for the defendant-appellant. |
| | CNN | *Predicted:* *1155 Davina T. Chen, Deputy Federal Public Defender, Los Angeles, CA, for the defendant-appellant. |
| 3 | CRF | *Ground Truth:* Decided: November 26, 2002. |
| | | *Predicted:* Decided: |
| 4 | CRF | *Ground Truth:* Veteran represented by: James McElfresh II, Agent. |
| | | *Predicted:* Veteran represented by: |
| 5 | CRF | *Ground Truth:* Appellant represented by: Harold H. Hoffman III, Attorney. |
| | | *Predicted:* Appellant represented by: |
| 6 | CRF | *Ground Truth:* Franklin also moved to dismiss eleven of the fourteen copyright infringement counts on the ground that Apple failed to comply with the procedural requirements for suit under 17 U. S. C. §§410, 411. ⟨ 714 F. 2 d 1245⟩ |
| | | *Predicted:* Franklin also moved to dismiss eleven of the fourteen copyright infringement counts on the ground that Apple failed to comply with the procedural requirements for suit under 17 U. S. C. §§410, 411. |
| 7 | CRF | *Ground Truth:* See VAOPGCPREC 82–90 at para. 2. |
| | | *Predicted:* See VAOPGCPREC 82–90 at para. |
| 8 | CNN | *Ground Truth:* United States v. Barsumyan, 517 F.3d 1154 (9th Cir. 2008) |
| | | *Predicted:* United States v. Barsumyan, 517 F.3d 1154 (9th Cir. 2008)Court of Appeals for the Ninth CircuitFiled: February 28th, 2008 Precedential Status: PrecedentialCitations: 517 F.3d 1154 Docket Number: 07-50251 Author: Joseph Jerome Farris 517 F.3d 1154 (2008) |

**Table 10** (continued)

| Sl. No.: | Model | Sample errors |
|---|---|---|
| 9 | CNN | *Ground Truth:* In October 2005, an informant cooperating with the United States Secret Service charged that Defendant-Appellant Aram Barsumyan was involved in schemes to manufacture counterfeit credit cards. |
| | | *Predicted:* Facts, Procedural History, and Jurisdiction In October 2005, an informant cooperating with the United States Secret Service charged that Defendant-Appellant Aram Barsumyan was involved in schemes to manufacture counterfeit credit cards. |
| 10 | Hybrid | *Ground Truth:* Evidence of Extraneous Offenses. |
| | | *Predicted:* V. Evidence of Extraneous Offenses. |
| 11 | Hybrid | *Ground Truth:* Ruben WENGER V a/k/a Reuben V. Wenger, Appellant v. The STATE of Texas, State |
| | | *Predicted:* The STATE of Texas, State |
| 12 | Hybrid | *Ground Truth:* 475 U. S., at 114–115. |
| | | *Predicted:* 7 475 U. S., at 114–115. |

**Table 11** Sample errors of the standard toolkit pySBD model in Section A and hybrid model in Section B in the context of Indian legal domain

| SECTION A: Sample errors in PySBD model |
|---|
| 1. *Errors due to confusing '-' as an end-of-sentence delimiter* |
| i. ***Ground Truth:*** section 12 of the Act is in these terms- A landlord shall not be entitled to the recovery of possession of any premises so long as the tenant pays, or is ready and willing to pay, the amount of the standard rent and permitted increases, if any, and observes and performs the other conditions of the tenancy, in so far as they are consistent with the provisions of this Act. ***Predicted:*** section 12 of the Act is in these terms- |
| ii. ***Ground Truth:*** This figure of Rs. 65,000 is arrived at by the following process - Gross income from rents. ***Predicted:*** This figure of Rs. 65,000 is arrived at by the following process - |
| 2. *Errors due to confusing ')' as an end-of-sentence delimiter* |
| i. ***Ground Truth:*** In sub-clause (2) (a) the definition is in these terms - The expression estate shall, in relation to any local area, have the same meaning as that expression or its local equivalent has in the existing law relating to land tenures in force in that area, and shall also include any jagir, inam or muafi or other similar grant. ***Predicted:*** In sub-clause (2) |
| ii. ***Ground Truth:*** (c) (d) (e) and (f) Similar heads of objections were formulated and argued at considerable length. ***Predicted:*** (c) |
| 3. *Errors due to confusing '.' followed by a small case letter used in the context of abbreviation* |
| i. ***Ground Truth:*** The land assessment on malguzari villages was it is alleged, raised to 75 per cent. from 45 to 50 per cent. of malguzari assets. ***Predicted:*** The land assessment on malguzari villages was it is alleged, raised to 75 per cent. |
| **SECTION B. Sample errors in Hybrid Model** |
| 4. *Errors due to confusing text structure:* |
| i. ***Ground Truth:*** The relevant portion of section 22 as amended runs as follows—22. (1) Any dealer aggrieved by an order under this Act may, in the prescribed manner, appeal to the prescribed authority against the order Provided that no appeal against an order of assessment, with or without penalty shall be admitted by the said authority unless such appeal is accompanied by a satisfactory proof of the payment of the tax, with penalty, if any, in respect of which the appeal has been preferred. ***Predicted:*** The relevant portion of section 22 as amended runs as follows—22. |
| ii. ***Ground Truth:*** The relevant portion of this resolution was as follows From ancient times there has been a tradition in our family to grant a village to the Maharani for her enjoyment in order to maintain her status and dignity. ***Predicted:*** The relevant portion of this resolution was as follows |
| 5. *Errors due to model encountering '.' followed by a capital letter.* |
| i. ***Ground Truth:*** In the Oxford New English Dictionary, Vol. 11, page 888, consumption is defined as The action or fact of consuming or destroying destruction (7) Pol. Econ. ***Predicted:*** In the Oxford New English Dictionary, Vol. 11, page 888, consumption is defined as The action or fact of consuming or destroying destruction (7) Pol. |

models faced challenges in accurately identifying character numerals at the beginning of sentences, as evident in samples 1 and 2 of Table 10. It is worth noting that these errors remained persistent in the hybrid model, as seen in sample 12. Specifically, the models struggled with identifying errors in relation to numerals at the beginning of sentences, potentially confusing them as enumerated lists. Samples 3, 4, and 5 highlighted the CRF models' struggles in correctly detecting boundaries using the delimiter ':'-an issue successfully addressed by the CNN model. However, the CNN model encountered difficulties with parentheses within sentences, erroneously considering them as

separate sentence markers, a problem resolved by our hybrid model. Furthermore, the baseline CRF models exhibited improper handling of citations within sentences, mistakenly treating them as separate sentences (Sanchez 2019), as illustrated in sample 6. Some specific citation formats, such as 'verses' templates (see samples 10 and 11), posed challenges for the hybrid model, causing confusion in boundary detection. The success of the CRF model heavily relied on the ingenuity applied in engineering its features, and incorporating CNN output features further improved the results. Thus, our hybrid model, integrating CNN features into CRF, rectifies errors and demonstrates a 4% overall improvement in F1-score compared to baseline models (see Table 7).

To assess the model's ability to generalize to new and unseen data distributions, the hybrid model's evaluation was also performed in the Indian Dataset. Table 9 highlights the hybrid-model performance with other standard toolkits when applied in an out-of-domain test set. The low performance of most toolkits, with the exception of pySBD, is evident in the table. PySBD stands out due to its non-destructive sentence tokenization approach, employing English golden rules (Sadvilkar and Neumann 2020). Although English serves as the official legal language in India, it often involves transcribing from native rural languages, leading to variations in meaning and the essence of legal concepts. Considering these factors, it is evident that Indian legal datasets differ from Western countries due to the unique cultural context, hierarchical structure, distinct legal terminology, divisions of statutory laws, and linguistic challenges. PySBD model often confuses '-' and ')' as end-of-sentence markers as seen in Table 11, examples 1 and 2. Also the Indian dataset has a lot of colloquial variations of the English language and does not follow all the rules of the standard English language. A good example is seen in example 3 of Table 11, where "cent." is a non-standard shortening, which is not in accordance with the standard English language. This error is significant as it leads to fragmented sentences being forwarded for additional processing within the pipeline. Our hybrid architecture was able to identify these errors listed in Section A of Table 11 and predict the actual ground truth. Our hybrid model showcases robustness and excellent generalization capabilities, outperforming the pySBD model by 8% in terms of the average F1-score. The hybrid model, although quite effective in predicting sentence boundaries, still has scope for improvement. In Table 11 refer to Section B, example 1, we can see the use of a delimiter along with an unusual sentence structure causes wrong prediction by the hybrid model. Similarly, in example 4 of Table 11, we can see that an indentation has been used as punctuation, while the model does not recognize this and considers it to be a new line leading to wrong predictions. Also, in certain examples, as seen last example of Table 11, the model confuses the '.' delimiter to be an end-of-sentence delimiter and does not recognize it as a punctuation used in the context of representing abbreviations as it has been trained with a lot of examples where a '.' delimiter followed by a capital letter is usually considered to be the end of a sentence and the start of a new one. This can be a scope of improvement by adding rules to customize our model based on the style of the language used in the dataset. Thus, the model's ability to surpass all baselines further emphasizes its effectiveness and suitability for effectively tackling legal sentence boundaries.

## 7 Conclusions and future work

In this paper, we have presented a deep learning approach to address the challenging problem of detecting sentence boundaries in legal text. Our approach leverages the context surrounding the delimiter for detecting sentence boundaries in the legal text as a sequence classification problem. We evaluated the performance of various deep learning models and identified the best-performing CNN model based on model size, F1 score, and inference time. We compared the performance of our models with the state-of-art CRF model. Our evaluation results demonstrate the superiority of our proposed deep learning approach over the baselines, with significant improvements in the F1 score. We also identified the complementary errors generated by the top-performing models to provide valuable insights into the limitations of current SBD approaches. To further improve the performance of our approach, we incorporated the features of the best-performing CNN model into a statistical CRF model to obtain a hybrid architecture that leverages the strengths of both models. Our hybrid model achieved an F1 score improvement of over 4% compared to the individual models, demonstrating the effectiveness of combining deep learning models with traditional statistical models. Also, our hybrid model exhibits its effectiveness and robustness in accurately detecting sentence boundaries in the context of Indian legal text compared to standard open-source libraries. This achievement highlights the model's capability to handle the unique linguistic and structural features inherent to Indian legal documents.

Our paper needs an assessment of how these tools impact downstream tasks, presenting an opportunity for future investigation. A subtle enhancement in this prepossessing step can significantly impact the performance of subsequent NLP applications. Conducting such an evaluation would be an evident direction for further research. While it is conceivable that minor disparities among the top-performing models may not significantly influence specific tasks, this study illuminates the nature of these variations, offering valuable insights. Another possible next step involves the integration of rule-based approaches into the existing architecture, specifically designed to address the drawn-out error samples. Additionally, exploring the incorporation of ensemble techniques presents a promising opportunity to enhance the SBD process further. Moreover, the current corpus of legal texts needs more representation in other areas like tax documents, dockets, and headnotes as well as the absence of multilingual datasets for training and evaluation. Indeed, the deep learning model employed in our research holds potential for cross-linguistic capabilities, which can be trained on any language because it works on character level and can scale to other languages. While the features in CRF are particularly tailored to the English Language, many of the features are also applicable to other languages, such as (isdigit, length, etc.,) and add custom features of the particular language into the proposed model, making it a valuable approach to extend it to other languages. The segmentation of sentences using our deep learning models is finely tuned at the character level, eliminating the necessity for language-specific tokenization processes. This flexibility

extends to languages lacking explicit sentence delimiters, like Thai, where each character can undergo a check to determine its delimiter status. Although the theoretical implications of the hybrid model over various languages are as mentioned above, empirical evidence would strengthen the value of the proposed method and its adoption by the wider community.

**Data availability** The dataset we used in our work is open for research purposes. The code used in this study was developed using open-source tools and libraries and is open for researchers to reproduce the experiments and build upon the research findings, potentially leading to new insights.

## Declarations

**Conflict of interest** All authors certify that they have no involvement in any firm or entity with any financial or non-financial interest in the materials covered in this document.

## References

Agarap AF (2018) Deep learning using rectified linear units (ReLU). arXiv preprint arXiv:1803.08375

Allen LE, Lysaght LJ (2015) Modern logic as a tool for remedying ambiguities in legal documents and analyzing the structure of legal documents' contained definitions. Logic in the Theory and Practice of Lawmaking, 383–407

Bahdanau D, Cho KH, Bengio Y (2015) Neural machine translation by jointly learning to align and translate. In: 3rd international conference on learning representations. ICLR 2015

Bhattacharya P, Paul S, Ghosh K, Ghosh S, Wyner A (2021) DeepRhole: deep learning for rhetorical role labeling of sentences in legal case documents. Artif Intell Law. https://doi.org/10.1007/s10506-021-09304-5

Bird S (2006) NLTK: the natural language toolkit. In: Proceedings of the COLING/ACL 2006 interactive presentation sessions, pp 69–72

Brugger T, Stürmer M, Niklaus J (2023) MultiLegalSBD: a multilingual legal sentence boundary detection dataset. arXiv:2305.01211

Chalkidis I, Androutsopoulos I (2017) A deep learning approach to contract element extraction. In: JURIX, vol 2017, pp 155–164

Chalkidis I, Fergadiotis M, Malakasiotis P, Aletras N, Androutsopoulos I (2020) LEGAL-BERT: the Muppets straight out of law school. In: Findings of the association for computational linguistics: EMNLP 2020, pp 2898–2904

Chen H, Pieptea LF, Ding J (2022) Construction and evaluation of a high-quality corpus for legal intelligence using semiautomated approaches. IEEE Trans Reliab 71(2):657–673

Chollampatt S, Ng HT (2018) A multilayer convolutional encoder-decoder neural network for grammatical error correction. In: Proceedings of the AAAI conference on artificial intelligence, vol 32

Chung J, Gulcehre C, Cho K, Bengio Y (2014) Empirical evaluation of gated recurrent neural networks on sequence modeling. arXiv preprint arXiv:1412.3555

Collobert R, Weston J, Bottou L, Karlen M, Kavukcuoglu K, Kuksa P (2011) Natural language processing (almost) from scratch. J Mach Learn Res 12:2493–2537

Deroy A, Ghosh K, Ghosh S (2023) Ensemble methods for improving extractive summarization of legal case judgements. Artif Intell Law. https://doi.org/10.1007/s10506-023-09349-8

Dietterich TG (1998) Approximate statistical tests for comparing supervised classification learning algorithms. Neural Comput 10(7):1895–1923

Dunning T (1994) Accurate methods for the statistics of surprise and coincidence. Comput Linguist 19(1):61–74

Francis WN, Kucera H (1979) Brown corpus manual. Lett Editor 5(2):7

Gers FA, Schmidhuber J, Cummins F (2000) Learning to forget: continual prediction with LSTM. Neural Comput 12(10):2451–2471

Gillick D (2009) Sentence boundary detection and the problem with the us. In: Proceedings of human language technologies: the 2009 annual conference of the North American chapter of the association for computational linguistics, companion volume: short papers, pp 241–244

Glaser I, Moser S, Matthes F (2021) Sentence boundary detection in German legal documents. In: ICAART (2), pp 812–821

Grefenstette G, Tapanainen P (1994) What is a word, what is a sentence?: problems of tokenisation

Griffis D, Shivade C, Fosler-Lussier E, Lai AM (2016) A quantitative and qualitative evaluation of sentence boundary detection for the clinical domain. In AMIA summits on translational science proceedings, vol 2016, p 88

Habernal I, Faber D, Recchia N, Bretthauer S, Gurevych I, Spiecker genannt Döhmann I, Burchard C (2023) Mining legal arguments in court decisions. Artif Intell Law. https://doi.org/10.1007/s10506-023-09361-y

Honnibal M, Montani I, Van Landeghem S, Boyd A (2020) spaCy: industrial-strength Natural Language Processing in Python

Jain D, Borah MD, Biswas A (2023) A sentence is known by the company it keeps: improving legal document summarization using deep clustering. Artif Intell Law. https://doi.org/10.1007/s10506-023-09345-y

Kingma DP, Ba J (2015) Adam: a method for stochastic optimization. In: Bengio Y, LeCun Y (eds) 3rd international conference on learning representations. ICLR 2015, San Diego, CA, USA, May 7–9, 2015, Conference Track Proceedings

Kiss T, Strunk J (2006) Unsupervised multilingual sentence boundary detection. Comput Linguist 32(4):485–525

Kudo T, Richardson J (2018) SentencePiece: a simple and language independent subword tokenizer and detokenizer for neural text processing. In: Proceedings of the 2018 conference on empirical methods in natural language processing: system demonstrations. Association for Computational Linguistics, pp 66–71

Lafferty J, McCallum A, Pereira FC (2001) Conditional random fields: probabilistic models for segmenting and labeling sequence data

Lavery UA (1921) The language of the law. Am Bar Assoc J 7(6):277–283

Lin Z, Feng M, Santos CNd, Yu M, Xiang B, Zhou B, Bengio Y (2017) A structured self-attentive sentence embedding. arXiv preprint arXiv:1703.03130

Liu DC, Nocedal J (1989) On the limited memory BFGS method for large scale optimization. Math Program 45(1–3):503–528

López R, Pardo TA (2015) Experiments on sentence boundary detection in user-generated web content. In: Computational linguistics and intelligent text processing: 16th international conference, CICLing 2015, Cairo, Egypt, April 14–20, 2015, Springer proceedings, Part I 16, pp 227–237

Malik V, Sanjay R, Nigam SK, Ghosh K, Guha SK, Bhattacharya A, Modi A (2021) ILDC for CJPE: Indian legal documents corpus for court judgment prediction and explanation. In: Proceedings of the 59th annual meeting of the association for computational linguistics and the 11th international joint conference on natural language processing (volume 1: long papers). Association for Computational Linguistics, Online, pp 4046–4062

Mandal A, Ghosh K, Ghosh S, Mandal S (2021) A sequence labeling model for catchphrase identification from legal case documents. Artif Intell Law. https://doi.org/10.1007/s10506-021-09296-2

Manning CD, Surdeanu M, Bauer J, Finkel JR, Bethard S, McClosky D (2014) The Stanford CoreNLP natural language processing toolkit. In: Proceedings of 52nd annual meeting of the association for computational linguistics: system demonstrations, pp 55–60

McNemar Q (1947) Note on the sampling error of the difference between correlated proportions or percentages. Psychometrika 12(2):153–157

Mikheev A (2000) Tagging sentence boundaries. In: 1st meeting of the North American chapter of the association for computational linguistics

Minixhofer B, Pfeiffer J, Vulić I (2023) Where's the point? Self-supervised multilingual punctuation-agnostic sentence segmentation. arXiv preprint arXiv:2305.18893

Okazaki N (2007) CRFsuite: a fast implementation of conditional random fields (CRFs)

Păiş V, Tufiş D (2021) Capitalization and punctuation restoration: a survey. Artif Intell Rev 55(3):1681–1722

Palmer DD, Hearst MA (1997) Adaptive multilingual sentence boundary disambiguation. Comput Linguist 23(2):241–267

Qi P, Zhang Y, Zhang Y, Bolton J, Manning CD (2020) Stanza: a Python natural language processing toolkit for many human languages. In: Proceedings of the 58th annual meeting of the association for computational linguistics: system demonstrations

Read J, Dridan R, Oepen S, Solberg LJ (2012) Sentence boundary detection: a long solved problem? In: Proceedings of COLING 2012. Posters, pp 985–994

Rehbein I, Ruppenhofer J, Schmidt T (2020) Improving sentence boundary detection for spoken language transcripts. In: Proceedings of the twelfth language resources and evaluation conference. European Language Resources Association, Marseille, France, pp 7102–7111

Reynar JC, Ratnaparkhi A (1997) A maximum entropy approach to identifying sentence boundaries. In: Proceedings of the fifth conference on applied natural language processing, pp 16–19

Riley M (1989) Some applications of tree-based modelling to speech and language. In: Speech and natural language: proceedings of a workshop held at Cape Cod, Massachusetts, October 15–18, 1989

Rudrapal D, Jamatia A, Chakma K, Das A, Gambäck B (2015) Sentence boundary detection for social media text. In: Proceedings of the 12th international conference on natural language processing, pp 254–260

Sadvilkar N, Neumann M (2020) PySBD: pragmatic sentence boundary disambiguation. In: Proceedings of second workshop for NLP open source software (NLP-OSS), pp 110–114

Sanchez G (2019) Sentence boundary detection in legal text. In: Proceedings of the natural legal language processing workshop 2019, pp 31–38

Saravanan M, Ravindran B (2010) Identification of rhetorical roles for segmentation and summarization of a legal judgment. Artif Intell Law 18:45–76

Savelka J, Walker VR, Grabmair M, Ashley KD (2017) Sentence boundary detection in adjudicatory decisions in the United States. Trait Autom Lang 58:21

Schweter S, Ahmed S (2019) Deep-EOS: general-purpose neural networks for sentence boundary detection. In: KONVENS

Sheik R, Gokul T, Nirmala S (2022) Efficient deep learning-based sentence boundary detection in legal text. In: Proceedings of the natural legal language processing workshop 2022, pp 208–217

Sutton C, McCallum A et al (2012) An introduction to conditional random fields. Found Trends® Mach Learn 4(4):267–373

Vaswani A, Shazeer N, Parmar N, Uszkoreit J, Jones L, Gomez AN, Kaiser Ł, Polosukhin I (2017) Attention is all you need. In:Proceedings of the 31st International Conference on Neural Information Processing Systems. NIPS'17, pp 6000–6010

Wang X, Utiyama M, Sumita E (2019) Online sentence segmentation for simultaneous interpretation using multi-shifted recurrent neural network. In: Proceedings of machine translation summit XVII: research track, pp 1–11

Wicks R, Post M (2021) A unified approach to sentence segmentation of punctuated text in many languages. In: Proceedings of the 59th annual meeting of the association for computational linguistics and the 11th international joint conference on natural language processing (volume 1: long papers), pp 3995–4007

Wong F, Chao S (2010) iSentenizer: an incremental sentence boundary classifier. In: Proceedings of the 6th international conference on natural language processing and knowledge engineering (NLPKE-2010). IEEE, pp 1–7

Young T, Hazarika D, Poria S, Cambria E (2018) Recent trends in deep learning based natural language processing. IEEE Comput Intell Mag 13(3):55–75

Zheng L, Guha N, Anderson BR, Henderson P, Ho DE (2021) When does pretraining help? Assessing self-supervised learning for law and the casehold dataset of 53,000+ legal holdings. In: Proceedings of the eighteenth international conference on artificial intelligence and law, pp 159–168

**Publisher's Note** Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.